Journal of Machine Learning Research 1 (2026) 1-25 Submitted 4/26; Published –

Sparse-Graph Fusion Solvers for Scalable Grouped Regression

Daniel Agyapong AGYAPONGDANIEL7777@GMAIL.COM
Northern Arizona University
Flagstaff, AZ, USA

Editor:

Abstract

Grouped regression models estimate related but subgroup-specific linear parameters by combining sparsity with fusion penalties across a graph of groups. Existing implementations can become expensive when the number of groups is large, especially when they materialize full pairwise fusion operators even though the scientific or data-driven fusion graph is sparse. This paper presents a sparse-graph representation principle for grouped fused regression: if the statistical objective is defined on a sparse graph, the optimizer should preserve the same edge set rather than materializing a complete graph and encoding sparsity through zeros. This principle gives both an equivalence guarantee for the exact solvers and an immediate complexity reduction from all-pairs graph construction to active-edge graph construction. The `fuserplus`package implements this principle for L1- and L2-fused grouped regression. The L1 operator solver evaluates the same smoothed proximal-gradient updates using only active graph edges, while the chain-specialized solver replaces a general graph by a DFS-induced chain when maximum computational efficiency is preferred over exact graph fidelity. The L2 solver reformulates the augmented regression design using active-edge triplets rather than dense all-pair construction before calling `glmnet`. Across public grouped regression benchmarks, the exact L1 operator reproduces the legacy predictive accuracy while reducing median runtime by 2.26x and median peak memory by 10.7% on the completed benchmark suite. The chain variant gives a median 3.26x runtime reduction with an accuracy tradeoff. Across 19 L2 benchmark datasets, active-edge construction has no systematic loss in test error and reduces median fit time by 14.9x. The implementation includes package tests, documented solver routines, benchmark manifests, and the CSV summaries used to regenerate the paper figures.

Keywords: fused lasso, grouped regression, sparse graphs, optimization, R package

1 Introduction

Many regression problems contain a natural grouping variable: patients may belong to disease subtypes, schools to districts, countries to regions, or time series to meters. Fitting a separate model in each group ignores shared signal, whereas fitting one global model ignores heterogeneity. Grouped fused regression addresses this middle ground by estimating one coefficient vector per group and penalizing differences between coefficient vectors along a graph of group relationships.

Let $X_g \in \mathbb{R}^{n_g \times p}$ and $y_g \in \mathbb{R}^{n_g}$ denote the data for group g, and let $\beta_g \in \mathbb{R}^p$ be that group's linear coefficient vector. For a graph $G = (V, E)$ over k groups with nonnegative

edge weights w_{gh}, the estimators considered in this paper have the form

$$\min_{\beta_1,\ldots,\beta_k} \sum_{g=1}^{k} \frac{1}{2n_g} \|y_g - X_g\beta_g\|_2^2 + \lambda \sum_{g=1}^{k} \|\beta_g\|_1 + \gamma \sum_{(g,h)\in E} w_{gh}\Omega(\beta_g - \beta_h). \tag{1}$$

For the L1 fused model, $\Omega(u) = \|u\|_1$; for the L2 fused model, $\Omega(u) = \frac{1}{2}\|u\|_2^2$ after the standard augmented-design reformulation. Optional group-specific intercepts are included in the software implementation but are excluded from sparsity and fusion penalties. This formulation is a direct descendant of the lasso, fused lasso, grouped and elastic-net regularization, graph-structured fused optimization, and subgroup regression models reviewed below. The L1 solver uses the standard Nesterov smoothing strategy for nonsmooth penalties (Nesterov, 2005).

The central computational issue is a mismatch between the statistical object and the solver representation. A complete graph has $k(k-1)/2$ edges, but practical graphs are frequently chains, trees, nearest-neighbor graphs, or domain-informed sparse networks. If the objective is defined on only m active edges, a solver that builds all pairwise fusion constraints and then uses zero weights for inactive edges changes the computational scaling of the problem without changing its statistical content. This matters as k grows: the modeling assumption is sparse, but the legacy representation is still quadratic.

The contribution of this paper is to make the sparse graph a first-class object in the optimizer. The work is computational rather than statistical: it does not propose a new penalty or estimator. Instead, it shows that the same estimator can be represented in a way that respects the graph on which the penalty is defined. First, it gives an exact active-edge L1 operator form that evaluates the smoothed proximal-gradient fusion term only over active edges. Second, it gives an exact active-edge L2 augmented-design construction that emits sparse triplets only for nonzero graph edges before delegating the regularized regression path to `glmnet` (Friedman et al., 2010). Third, it separates these exact reformulations from approximation-oriented solver modes, including a chain-specialized L1 approximation that collapses a graph to $k-1$ chain edges using a depth-first traversal, motivated by chain-structured fused lasso solvers. The result is a reusable design pattern for multi-task fused regression and other graph-penalized models whose penalties decompose over graph edges.

The main contributions are:

- a sparse-graph representation principle for grouped fused regression solvers;
- exact active-edge L1 and L2 reformulations that preserve the legacy objective while avoiding zero-weight all-pair construction;
- a clear separation between exact sparse-graph solvers and approximate high-efficiency chain variants;
- empirical comparisons showing when fused regression improves over unfused `glmnet` baselines and when active-edge construction improves over legacy fused solvers; and
- an open R implementation with benchmark scripts, manifests, and figure-generation artifacts.

The implementation is provided in the R package `fuserplus`. The current benchmark suite uses public grouped regression datasets and synthetic scaling experiments to compare prediction error, wall time, and peak resident memory. The results show that graph-aware construction can deliver substantial runtime and memory reductions without changing the statistical objective for the exact operator and L2 variants.

2 Related Work

The fused lasso extends the lasso by combining sparsity with penalties on ordered coefficient differences (Tibshirani, 1996; Tibshirani et al., 2005). This places grouped fused regression in a broader family of structured regularizers, including the elastic net, group lasso, generalized lasso, and graph trend filtering (Zou and Hastie, 2005; Yuan and Lin, 2006; Tibshirani and Taylor, 2011; Wang et al., 2016). Specialized algorithms for one-dimensional signal approximation, including path algorithms for the fused lasso signal approximator, exploit the ordered chain structure of that problem (Hoefling, 2010). Split Bregman and ADMM methods provide another important optimization line for nonsmooth fused penalties (Ye and Xie, 2011; Boyd et al., 2011). General graph fusion is more flexible but computationally harder because the penalty couples many coefficient blocks.

Graph-structured multi-task regression extended these ideas to arbitrary task graphs and developed smoothed proximal-gradient methods for the general fused lasso (Chen et al., 2010, 2012a). Related multi-task and scientific applications use graph-guided fusion for eQTL mapping, additive modeling, gene regulatory network inference, and subgroup-specific regression (Chen et al., 2012b; Petersen et al., 2016; Omranian et al., 2016). Network lasso methods (Hallac et al., 2015), graph trend filtering, DFS fused lasso denoising (Tansey et al., 2018), and k-nearest-neighbor fused lasso estimators (Padilla et al., 2020) further emphasize that the graph itself is part of the modeling problem: it may come from prior knowledge, distances between groups, or other data-driven structure.

The present work differs from these papers in emphasis. It does not introduce a new statistical penalty. Instead, it asks how existing grouped fused regression objectives should be represented and solved when the graph is sparse. This is more than an implementation detail: the graph is part of the modeling assumption, and a dense all-pairs representation discards that assumption at the computational layer. The active-edge constructions developed below restore the intended edge-level representation while preserving the legacy objective in the exact L1 and L2 cases.

Table 1 summarizes the closest available software ecosystem. The closest package overall is the R package `fuser`, because it implements the same subgroup-regression interface and L1/L2 fusion family that `fuserplus`extends. Outside R, the closest package by statistical model is the MATLAB MMT library for graph-guided multi-task lasso, which uses coefficient matrices and explicit task graphs but targets multi-task learning rather than the `X`, `y`, `groups`, `G` subgroup API. The closest Python package by penalty language is `yaglm`, which supports structured penalties including fused and generalized lasso, but is a general penalized-GLM framework rather than a subgroup-fusion regression package. Packages such as `genlasso` and `Lasso.jl` overlap with the fused/generalized lasso optimization language, but not with the paired L1/L2 subgroup-fusion workflow considered here. Additional packages provide useful but narrower reference points: `flsa` targets fused lasso signal

Table 1: Closest software relatives to `fuserplus`. The main distinction is that `fuserplus`targets subgroup regression with inputs (X, y, groups, G), both L1 and L2 fusion, and sparse-graph solver variants for scaling.

Package	Language	Closest overlap	Difference from `fuserplus`
`Lasso.jl` (JuliaStats, 2026)	Julia	Lasso paths plus fused lasso and trend filtering support.	Primarily regression paths, one-dimensional fused lasso, and trend filtering; not a graph-based subgroup-fusion package.
MMT (Chen, 2012)	MATLAB	Graph-guided multi-task lasso with coefficient matrices and task graphs.	Multi-task learning library rather than subgroup-regression API; includes feature-graph and task-graph fusion with a different optimizer.
`CVXPY` (Diamond and Boyd, 2016)	Python	Convex modeling language with total-variation atoms and custom fused penalties.	Useful for prototyping formulations; not a packaged subgroup-fusion estimator or benchmarked solver workflow.
`yaglm` (Janizek and Lee, 2021)	Python	General penalized-GLM framework with structured penalties including fused lasso.	Closest Python penalty toolbox, but not a dedicated subgroup L1/L2 fusion package.
`extlasso` (Ghosh and Chatterjee, 2026)	R	Fused-lasso penalized linear regression with λ_1 and λ_2 penalties.	Fuses neighboring coefficients in a single regression model; not graph fusion across subgroup-specific coefficient vectors.
`flsa` (Hoefling, 2024)	R	Path algorithm for the fused lasso signal approximator, including connected difference structures.	Signal-approximation focus with response vector or image-like inputs; not subgroup coefficient fusion for regression predictors.
`fuser` (Dondelinger and Mukherjee, 2017; Dondelinger et al., 2020)	R	Same subgroup fused-regression family; same legacy L1/L2 solver interface.	Closest overall baseline; `fuserplus`adds active-edge and chain variants focused on sparse-graph runtime and memory scaling.
`genlasso` (Arnold and Tibshirani, 2025)	R	Generalized-lasso and fused-lasso path algorithms.	General path solver; does not provide the subgroup-regression package workflow or paired L1/L2 fusion API.
`penalized` (Goeman, 2026)	R	L1, fused-lasso, and L2 penalized estimation for GLMs and Cox models.	Broad penalized-model package; not designed around subgroup graphs or active-edge L1/L2 fusion construction.
`smoothedLasso` (Reid and Tibshirani, 2021)	R	Nesterov-smoothed L1 penalty operators, including fused-lasso helpers.	Penalty/operator framework rather than a grouped-regression package with benchmarked L1/L2 sparse-graph solvers.

approximation, `extlasso` provides fused-lasso penalized linear regression, `smoothedLasso` implements smoothed L1 penalty operators, `penalized` supports fused-lasso penalties in several regression families, and `CVXPY` can express total-variation or fused penalties through convex modeling atoms. Citations for each package are given directly in the table.

3 Model and Legacy Computation

The L1 solver in `fuserplus`follows a smoothed proximal-gradient strategy for Equation 1. Let $B = [\beta_1, \ldots, \beta_k]$ be the $p \times k$ coefficient matrix. The legacy implementation builds a constraint matrix with columns for sparsity and fusion constraints, then uses a smoothed dual approximation to compute the gradient contribution of the nonsmooth penalty. The

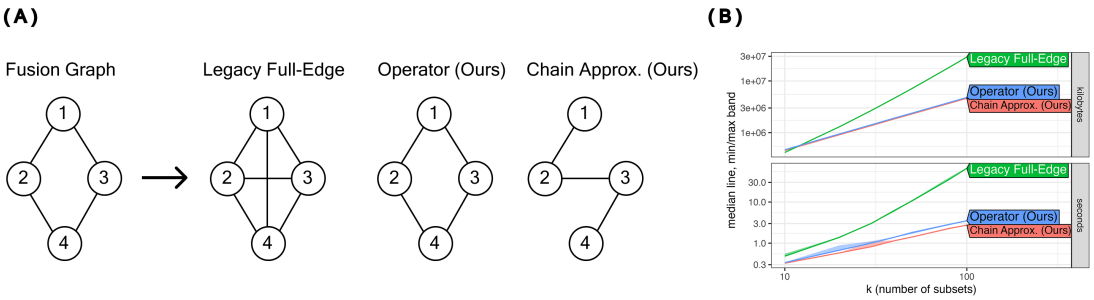

Figure 1: Conceptual overview of L1 sparse-graph fusion. A: fusion graph construction. The legacy full-edge construction associated with Dondelinger et al. (2020) forms a combined constraint matrix over all group pairs, while the active-edge operator works directly on the sparse graph. B: asymptotic complexity. The synthetic example fixes $p = 120$ and $n_{\text{group,train}} = 35$, so at $k = 100$ the total training sample size is $n = 3500$. With a sparse chain graph and fixed remaining hyperparameters, the active-edge representation changes the dominant fusion cost from $O(pk^3)$ to $O(pm)$.

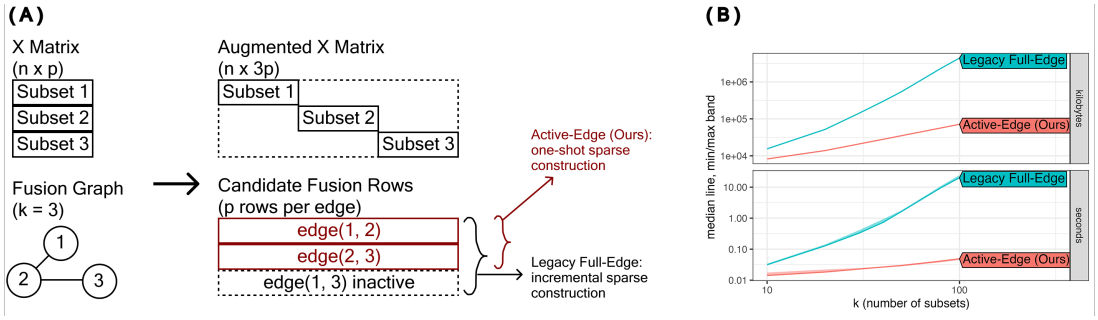

Figure 2: Conceptual overview of L2 sparse-graph fusion. A: augmented matrix construction. The legacy full-edge construction associated with Dondelinger et al. (2020) forms augmented rows for all group pairs, while the active-edge builder emits only sparse triplets for graph edges with nonzero weight. B: asymptotic complexity. The synthetic example fixes $p = 120$ and $n_{\text{group,train}} = 35$, so at $k = 100$ the total training sample size is $n = 3500$. With a sparse chain graph and fixed remaining hyperparameters, active-edge construction changes the dominant fusion-block cost from $O(pk^2)$ to $O(pm)$.

update has the generic accelerated form

$$A^\star = \Pi_{[-1,1]}\left(\frac{BC}{\mu}\right), \tag{2}$$

$$\nabla f_\mu(B) = \nabla \ell(B) + A^\star C^\top, \tag{3}$$

$$B^{t+1} = W^t - L^{-1}\nabla f_\mu(B^t), \tag{4}$$

where C encodes the sparsity and fusion operators, μ is the smoothing parameter, L is a Lipschitz bound, and W^t is the accelerated iterate. When C contains all group pairs, storage and multiplication include $O(pk^2)$ fusion terms for a complete representation.

More explicitly, the L1 implementation uses the dual identity $|z| = \max_{|a|\le 1} az$ and replaces the nonsmooth absolute-value terms by clipped smoothed duals. For sparsity and active fusion edges this gives

$$A_s^\star = \mathrm{clip}\left(\frac{\lambda B_s}{\mu}, -1, 1\right), \qquad A_{uv}^\star = \mathrm{clip}\left(\frac{\gamma w_{uv}(\beta_u - \beta_v)}{\mu}, -1, 1\right),$$

where B_s excludes intercept rows. The code uses the step-size upper bound

$$L_U = \lambda_{\max} + \frac{\lambda^2 + 2\gamma^2 d_{\max}^{(w)}}{\mu},$$

with $\lambda_{\max} = \sum_g \lambda_{\max}(X_g^\top X_g/n_g)$ under scaled group losses and weighted maximum degree proxy $d_{\max}^{(w)}$ for the active fusion graph.

Proposition 1 (Equivalence of dense and active-edge L1 gradients)
Fix a weighted graph G and let E denote the edges with positive weight. For smoothing parameter $\mu > 0$, suppose the dense smoothed proximal-gradient implementation includes one fusion column for every pair but assigns zero weight to inactive pairs. The active-edge operator implementation computes exactly the same smoothed fusion gradient as the dense implementation after removing the zero-weight columns.

Proof For edge $e = (u_e, v_e, w_e)$, the dense fusion column has nonzero endpoint entries with opposite signs. Its smoothed dual contribution is $p_e = \mathrm{clip}((\gamma w_e/\mu)(\beta_{u_e} - \beta_{v_e}), -1, 1)$. Multiplying the dense constraint matrix by this dual vector adds $\gamma w_e p_e$ to column u_e and subtracts $\gamma w_e p_e$ from column v_e. Algorithm 2 performs exactly these two scatter operations for each active edge and performs no operation for inactive zero-weight columns. Summing over edges therefore gives the same gradient contribution. ■

The L2 solver uses a different reduction. It constructs an augmented response and design matrix,

$$\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}, \quad \tilde{X} = \begin{bmatrix} X_{\mathrm{block}} \\ X_{\mathrm{fusion}} \end{bmatrix},$$

where X_{block} is block diagonal over groups and X_{fusion} contains, for each group pair, rows proportional to

$$\sqrt{\gamma w_{gh}}(\beta_g - \beta_h).$$

The resulting lasso problem is passed to `glmnet`. A full all-pairs construction creates $O(pk^2)$ fusion rows even when only $m = |E|$ edges are active.

Proposition 2 (Equivalence of active-edge L2 augmented rows) *For a fixed graph G, penalty γ, and scaling convention, the active-edge L2 augmented design is equivalent to the legacy all-pairs augmented design after deleting all zero-weight pair rows and applying a row permutation.*

Algorithm 1 Legacy Full-Edge Smoothed APG (`old_l1`)

Require: grouped data, G, $\lambda, \gamma, \mu, T, \varepsilon$

Construct full fusion matrix $C_{\text{fusion}} \in \mathbb{R}^{k \times \binom{k}{2}}$
Construct $C = [\lambda I_k, \gamma C_{\text{fusion}}]$
Initialize $B^{(0)} = 0,\ W^{(0)} = 0,\ S^{(0)} = 0$
for $t = 1, \ldots, T$ **do**
    Compute $\nabla \ell(B^{(t-1)})$ groupwise
    $A^\star \leftarrow \text{clip}\Big([B_s^{(t-1)} C_s,\ B_f^{(t-1)} C_f]/\mu, -1, 1\Big)$
    $\nabla r_\mu(B^{(t-1)}) \leftarrow A^\star C^\top$
    $G_t \leftarrow \nabla \ell(B^{(t-1)}) + \nabla r_\mu(B^{(t-1)})$
    Compute L_U from the dense max-degree proxy
    Update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
    if $\left\| \tilde{B}^{(t)} - B^{(t-1)} \right\|_1 < \varepsilon p$ **then**
        break
    end if
    $B^{(t)} \leftarrow \tilde{B}^{(t)}$
end for
Soft-threshold non-intercept rows by λ / n_g
return $\hat{B}$

Proof Each nonzero edge (u, v) contributes p_{eff} augmented rows with $+\sqrt{\gamma w_{uv}}$ in coefficient block u and $-\sqrt{\gamma w_{uv}}$ in coefficient block v, with zero augmented response. The active-edge builder constructs exactly these rows using sparse triplets. Inactive pair rows have zero weight in the legacy construction and therefore add zero residual contribution to the augmented least-squares objective. Removing them and permuting the remaining rows leaves the objective minimized by `glmnet` unchanged. ■

Proposition 3 (Representation-dependent graph cost)
Let m be the number of nonzero graph edges and let $q = \binom{k}{2}$. In one L1 smoothed-gradient evaluation, a dense all-pairs operator performs fusion work proportional to pq after construction, whereas the active-edge operator performs fusion work proportional to pm. In the L2 augmented-design construction, the fusion block decreases from pq rows to pm rows.

Proof For L1 fusion, each edge contributes one p-dimensional coefficient difference and one p-dimensional scatter back to the incident groups. A dense pairwise representation contains q candidate edges, including zero weight inactive pairs, while the active-edge representation iterates only over the m nonzero edges. For L2 fusion, each edge contributes p_{eff} augmented rows, one for each penalized coefficient coordinate. Deleting zero-weight pairs therefore reduces the fusion block from $p_{\text{eff}} q$ rows to $p_{\text{eff}} m$ rows, up to the intercept convention. ■

4 Sparse-Graph Solvers

4.1 Exact L1 Operator Solver

The exact L1 operator solver replaces the dense fusion matrix representation with edge lists $(u_e, v_e, w_e)_{e=1}^m$. At each iteration, the solver computes the fusion differences only on active edges,

$$D_e = \beta_{u_e} - \beta_{v_e}, \quad P_e = \Pi_{[-1,1]}\left(\frac{\gamma w_e D_e}{\mu}\right),$$

and scatters $\gamma w_e P_e$ back to the incident group columns with opposite signs. The sparsity part of the update remains unchanged. This representation has the same objective and first-order updates as the active-edge version of the legacy smoothed proximal-gradient solver, but the fusion work scales with pm rather than pk^2.

The implementation also supports block processing of edges. This is a practical memory safeguard for large p or m, since intermediate edge-difference matrices can otherwise dominate the working set. The operator solver is therefore intended as the general exact path for sparse graphs.

Active-edge gradient assembly. The operator implementation initializes $\Delta_f = 0$ and loops over active edges $e = 1, \ldots, m$. For each edge, it computes $d_e = \beta_{u_e} - \beta_{v_e}$ and $p_e = \mathrm{clip}((\gamma w_e/\mu)d_e, -1, 1)$, then performs the endpoint updates

$$\Delta_f(:, u_e) \leftarrow \Delta_f(:, u_e) + \gamma w_e p_e, \qquad \Delta_f(:, v_e) \leftarrow \Delta_f(:, v_e) - \gamma w_e p_e.$$

The accumulated matrix Δ_f is the fusion contribution added to the likelihood and sparsity gradients in the accelerated update.

Optimization algorithm. The implementation used in the experiments is the following active-edge APG procedure. It is the central exact L1 routine in `fuserplus`.

4.2 Working-Set Active-Edge Variant

The package also includes a working-set extension of the active-edge operator. This variant maintains an active edge set $A \subseteq E$, solves the smoothed problem on A, then scores inactive edges by

$$s_e = \gamma w_e \left\|\beta_{u_e} - \beta_{v_e}\right\|_\infty .$$

The highest-scoring inactive edges are added in later outer rounds. An optional screening rule scores zero-point likelihood-gradient gaps, $\|\nabla \ell_u(0) - \nabla \ell_v(0)\|_\infty$, and can drop edges whose score is below a margin-adjusted fusion threshold. This method is treated as an implementation variant rather than the main benchmark method, because its behavior depends on the screening and expansion schedule.

4.3 Chain-Specialized L1 Approximation

The chain-specialized solver is a deliberately approximate method. It builds a depth-first traversal of a maximum spanning tree or of the thresholded graph, reorders the groups along that traversal, and keeps only the $k-1$ adjacent chain edges. The fusion operator can then

Algorithm 2 Active-Edge Operator APG (`operator`)

Require: grouped data, edge list E, $\lambda, \gamma, \mu, T, \varepsilon$

Initialize $B^{(0)} = 0$, $W^{(0)} = 0$, $S^{(0)} = 0$
for $t = 1, \ldots, T$ **do**
  Compute groupwise likelihood gradient $\nabla \ell(B^{(t-1)})$
  $A_s^\star \leftarrow \mathrm{clip}(\lambda B_s^{(t-1)}/\mu, -1, 1)$
  Initialize fusion accumulator $\Delta_f \leftarrow 0$
  for each edge $e = (u_e, v_e, w_e) \in E$ **do**
    $d_e \leftarrow \beta_{u_e}^{(t-1)} - \beta_{v_e}^{(t-1)}$
    $p_e \leftarrow \mathrm{clip}((\gamma w_e/\mu) d_e, -1, 1)$
    $\Delta_f(:, u_e) \leftarrow \Delta_f(:, u_e) + \gamma w_e p_e$
    $\Delta_f(:, v_e) \leftarrow \Delta_f(:, v_e) - \gamma w_e p_e$
  end for
  $G_t \leftarrow \nabla \ell(B^{(t-1)}) + \lambda A_s^\star + \Delta_f$
  Compute L_U on active graph; update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
  if $\left\| \tilde{B}^{(t)} - B^{(t-1)} \right\|_1 < \varepsilon p$ **then**
    break
  end if
  $B^{(t)} \leftarrow \tilde{B}^{(t)}$
end for
Soft-threshold non-intercept rows by λ/n_g
return $\hat{B}$

be computed with vectorized adjacent differences:

$$D_j = \beta_j - \beta_{j+1}, \quad j = 1, \ldots, k-1.$$

The vectorized implementation forms

$$D = B_{:,1:(k-1)} - B_{:,2:k}, \qquad P = \mathrm{clip}((\gamma/\mu) D \,\mathrm{diag}(w), -1, 1),$$

and accumulates $\gamma P \,\mathrm{diag}(w)$ on neighboring chain endpoints. This reduces the edge budget to the minimum needed to connect the groups. It does not optimize the same objective as the original graph unless the graph is already the selected chain. It is useful when the main objective is maximum speed and memory reduction and the user is willing to accept a small loss in graph fidelity.

Optimization algorithm. The chain-specialized code first builds the DFS/MST order and adjacent chain weights, then runs a separate vectorized APG routine rather than calling the general edge loop:

4.4 Active-Edge L2 Augmented Design

The L2 active-edge solver preserves the augmented-design strategy but changes how the design is built. Instead of allocating all pairwise fusion blocks and then assigning zero-weight rows, it finds the upper-triangular nonzero entries of G and builds sparse triplets

Algorithm 3 Chain-Specialized APG (`chain_specialized`)

Require: grouped data, G, chain settings, $\lambda, \gamma, \mu, T, \varepsilon$

Build DFS/MST chain order π and adjacent chain weights $(w_1, \ldots, w_{k-1})$
Reorder groups by π; initialize $B^{(0)} = 0,\ W^{(0)} = 0,\ S^{(0)} = 0$
for $t = 1, \ldots, T$ **do**
    Compute groupwise likelihood gradient $\nabla \ell(B^{(t-1)})$
    $A_s^\star \leftarrow \mathrm{clip}(\lambda B_s^{(t-1)}/\mu, -1, 1)$
    $D \leftarrow B^{(t-1)}_{:,1:(k-1)} - B^{(t-1)}_{:,2:k}$
    $P \leftarrow \mathrm{clip}\Big(\frac{\gamma\, D\, \mathrm{diag}(w)}{\mu}, -1, 1\Big)$
    $C \leftarrow \gamma\, P\, \mathrm{diag}(w)$
    Initialize fusion accumulator $\Delta_f \leftarrow 0$
    $\Delta_f(:, 1:(k-1)) \mathrel{+}= C, \qquad \Delta_f(:, 2:k) \mathrel{-}= C$
    $G_t \leftarrow \nabla \ell(B^{(t-1)}) + \lambda A_s^\star + \Delta_f$
    Compute chain L_U and update $\tilde{B}^{(t)}, S^{(t)}, W^{(t)}$
    if $\left\| \tilde{B}^{(t)} - B^{(t-1)} \right\|_1 < \varepsilon p$ **then**
        break
    end if
    $B^{(t)} \leftarrow \tilde{B}^{(t)}$
end for
Soft-threshold non-intercept rows by λ / n_g
Reorder columns back to original group order
return $\hat{B}$

only for those edges. The same scaling semantics as the legacy implementation are used:

$$\sqrt{\gamma(n + n_{\text{fusion}})}\sqrt{w_{gh}}$$

multiplies the positive and negative entries in the fusion rows. The final augmented design is passed to `glmnet` with the same lambda correction factor as the original wrapper.

This change leaves the fitted objective unchanged for a given active graph but reduces the number of constructed fusion rows from $pk(k-1)/2$ to pm. The largest expected gains occur when $m \ll k(k-1)/2$.

Optimization algorithm. The L2 path uses `glmnet` after building an augmented sparse regression problem:

5 Experimental Design

5.1 Datasets

The repository contains a processed grouped-regression inventory of 22 public datasets. Across this inventory, the number of observations ranges from 200 to 220,908, the number of features from 3 to 100, and the number of groups from 10 to 3,058. Sources include UCI datasets, Rdatasets mirrors, Our World in Data CO2 data, World Bank indicators, NYC TLC trip records, OpenML, and other public sources documented in the repository.

Algorithm 4 L2 via Augmented Design and `glmnet`

Require: grouped data (X, y, groups), graph G, λ, γ, scaling settings
1: Build block-diagonal design X_{block} over groups
2: Build fusion rows $X_{\text{fusion}} \leftarrow \text{BuildFusionRows}(G)$
3: Scale fusion block by $\sqrt{\gamma(n + n_{\text{fusion}})}$
4: Form augmented system $\tilde{X} = [X_{\text{block}}; X_{\text{fusion}}]$, $\tilde{y} = [y; 0]$
5: Fit θ with `glmnet` on $(\tilde{X}, \tilde{y})$ using λ
6: Reshape θ to $B \in \mathbb{R}^{p_{\text{eff}} \times k}$
7: **return** B

Algorithm 5 BuildFusionRows(G) for Active-Edge L2

Require: graph G
1: Extract active edges $E = \{(u, v) : u < v,\ G_{uv} \neq 0\}$
2: Initialize sparse triplet lists for X_{fusion}
3: **for** each active edge $(u, v) \in E$ **do**
4: Add p_{eff} rows with $+\sqrt{w_{uv}}$ on block u
5: Add matching entries with $-\sqrt{w_{uv}}$ on block v
6: **end for**
7: **return** X_{fusion}

The completed L1 benchmark summary covers 12 datasets with five outer folds per dataset. The L2 benchmark summary covers 19 datasets with five outer folds per dataset, including 11 datasets with at least 50 groups. The reported figures and tables are generated from CSV summaries under `build/hpc` and `build/hpc/l2_knn`; the plotting artifacts are stored under `inst/figures` and copied into the paper source tree.

Table 3 lists the complete union of datasets that appear in the final L1 and L2 result summaries used by this paper. The remaining processed datasets are retained in the repository for future experiments but are not included in the reported final benchmark panels. The `SubsetType` column reports the semantic unit defining the subgroups in the regression problem.

5.2 Evaluation Protocol

All reported real-data results use grouped outer cross-validation with five stratified folds and random seed 20260221. Splits are made at the observation level while preserving group representation where possible; datasets with very small groups are filtered using a minimum group size of seven. All graph construction, hyperparameter selection, and standardization are performed inside the outer-training fold so that the held-out test fold is not used in model selection.

For each outer fold, the baseline sparsity parameter is selected on the outer-training data by `cv.glmnet` with three inner folds and `lambda.min`. A separate inner train-validation split of the outer-training data selects the fusion strength γ by validation RMSE. The final model is then refit on the full outer-training fold using the selected λ and γ and evaluated on the held-out outer fold. The featureless baseline predicts the training-fold mean for the

Table 2: Dominant representation costs of the solver variants. Here n is total sample size, p is feature count, k is group count, $m = |E|$ is the number of active fusion edges, $q = \binom{k}{2}$, T is the L1 iteration count, and I is the number of `glmnet` sweeps. The table isolates graph-representation costs and omits lower-order constants from R bookkeeping and solver overhead.

Method	Time	Memory
legacy L1 full edge	$O(T(np + pkq))$	$O(np + pk + pq + kq)$
L1 active-edge operator	$O(T(np + pm))$	$O(np + pk + m)$
L1 chain-specialized	$O(T(np + pk))$	$O(np + pk)$
legacy L2 full pairwise	$O(I(np + pq))$	$O(np + pq)$
L2 active-edge	$O(I(np + pm))$	$O(np + pm)$

corresponding response. Runtime is measured for the fit stage, and memory is the peak resident set size recorded by the worker process.

5.3 Graph Construction and Hyperparameters

The real-data benchmark constructs a data-driven group graph from training data, following the use of graph-based relationships and k-nearest-neighbor fusion graphs in prior work (Hallac et al., 2015; Padilla et al., 2020). Group centroids are computed on the outer-training fold, a k-nearest-neighbor graph is formed from centroid distances, inverse-distance weights are used, and connectivity is enforced by adding nearest inter-component links. The manifest records $k = 5$ nearest neighbors, inverse-distance weights, and MST-style connectivity augmentation. This follows the general idea that graph structure should encode similarity between groups rather than defaulting to a complete graph.

Each dataset is evaluated with five outer folds. The regularization parameter λ is warm-started from `cv.glmnet`; the fusion parameter γ is selected from the grid

$$10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}.$$

The L1 optimization tolerance is 10^{-5} with a maximum of 1200 iterations in the HPC manifest. Runs use intercepts for L1 models, no feature scaling inside the L1 solver, standardized inputs for benchmark construction, an edge-block size of 256 for the operator implementation, and the adaptive operator fallback threshold recorded in the manifest. The L2 panel uses the same outer-fold and graph protocol with the L2 solvers.

5.4 Compared Methods

For L1 fusion, the benchmark compares the legacy full construction (`old_l1`), the exact active-edge operator solver (`operator`), the chain approximation (`chain_specialized`), and a featureless baseline. For L2 fusion, the benchmark compares the legacy augmented design (`old_l2`), the active-edge augmented design (`new_l2`), and the featureless baseline.

The exact comparisons are intentionally conservative. The operator and active-edge L2 methods are compared against legacy implementations using the same graph, outer folds,

Table 3: Public grouped-regression datasets used in the reported real-data benchmark panels. The columns report total observations n, feature count p, group count k, and semantic subgroup type after preprocessing in this repository.

Dataset	n	p	k	`SubsetType`	Source
`beijing_air_quality`	180,000	14	12	stations	UCI Beijing air quality (Chen, 2017)
`communities_crime`	1,991	100	43	states	UCI Communities and Crime (Redmond, 2002)
`electricity_load`	200,000	3	120	meters	UCI Electricity Load (Trindade, 2015)
`nyc_tlc_yellow_2022`	200,000	9	82	pickup zones	NYC TLC yellow taxi records (New York City Taxi and Limousine Commission, 2022)
`owid_co2`	7,763	4	164	countries	Our World in Data CO2 (Ritchie et al., 2023)
`radon_minnesota`	916	5	82	counties	PyMC Minnesota radon example (PyMC Development Team, 2026)
`rdatasets_cigar`	1,380	7	46	states	Rdatasets, `plm::Cigar` (Arel-Bundock, 2026)
`rdatasets_crime4`	630	57	90	counties	Panel crime data (Cornwell and Trumbull, 1994)
`rdatasets_dietox`	861	6	72	pigs	Dietox pig-growth data (Lauridsen et al., 1999)
`rdatasets_empluk`	1,031	5	140	firms	Employment panel data (Arellano and Bond, 1991)
`rdatasets_exam_inner_london`	4,057	7	64	schools	Inner London exam data (Goldstein et al., 1993)
`rdatasets_fatalities`	336	32	48	states	Rdatasets, `AER::Fatalities`
`rdatasets_grunfeld`	200	3	10	firms	Rdatasets, `plm::Grunfeld`
`rdatasets_guns`	1,173	11	51	states	Rdatasets, `AER::Guns`
`rdatasets_hsb82`	7,185	6	160	schools	High-school multilevel data (Raudenbush and Bryk, 2002)
`rdatasets_ratpupweight`	322	3	27	litters	Rdatasets, `nlme::RatPupWeight`
`sarcos_openml`	44,484	21	10	response bins	OpenML SARCOS 43873 (OpenML, 2026)
`school_ilea`	7,185	10	160	schools	`nlme` school data (Pinheiro and Bates, 2000)
`world_bank_wdi`	10,830	4	229	economies	World Development Indicators (World Bank, 2026)

selected hyperparameters, and prediction pipeline. Thus the reported differences isolate representation and construction costs, not changes in the statistical model. The chain-specialized L1 method is reported separately because it changes the graph and is therefore an approximation rather than an exact replacement.

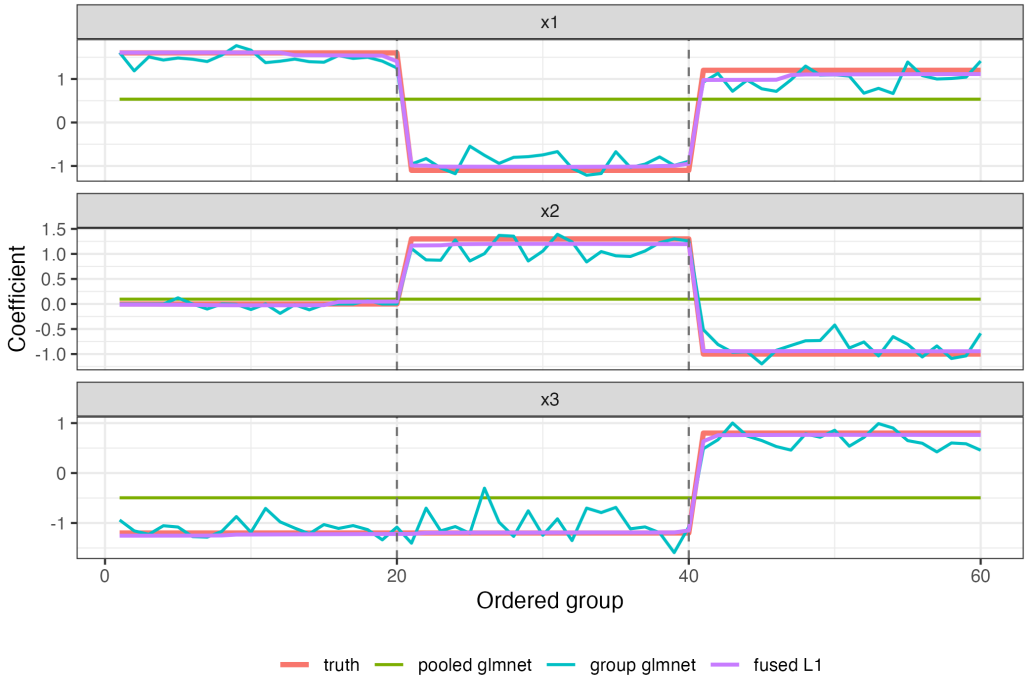

Figure 3: Controlled changepoint example comparing fused regression with `glmnet` baselines. Dashed vertical lines mark the true changepoints. Pooled `glmnet` underfits group heterogeneity, group-specific `glmnet` overfits noisy group-level paths, and the fused L1 model recovers the piecewise-constant coefficient structure.

6 Results

6.1 Changepoint Structure Relative to Glmnet

Before comparing solver representations, it is useful to show a setting where fusion is statistically useful relative to unfused `glmnet` baselines. We generated an ordered-group regression problem with $k = 60$ groups, $p = 20$ predictors, and piecewise-constant coefficient paths with changepoints after groups 20 and 40. Each group has 25 training observations for fitting, 10 validation observations for tuning, and 80 test observations. The graph is the chain over ordered groups. We compare a pooled `glmnet` model, a group-specific interaction `glmnet` model with no fusion, and the L1 fused model on the same chain graph.

The pooled `glmnet` baseline cannot represent group-specific changepoints. The group-specific `glmnet` baseline can represent heterogeneity but has no mechanism to share strength across adjacent groups, so its fitted paths are noisy and contain many spurious jumps. The fused L1 model uses the graph penalty to recover the piecewise-constant structure. In this controlled example it has the lowest test RMSE and coefficient error, and it produces exactly two large jumps, both at the true changepoints.

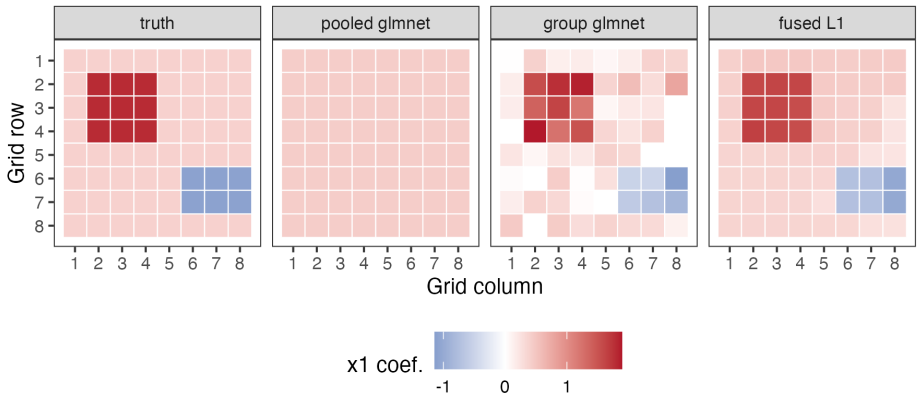

Figure 4: Controlled spatial-hotspot example comparing fused regression with `glmnet` baselines on an 8×8 group grid. The panels show the first coefficient over the spatial graph. Pooled `glmnet` smooths away the hotspots, group-specific `glmnet` gives noisy fragmented regions, and fused L1 recovers the contiguous hotspot structure.

6.2 Spatial Hotspot Structure Relative to Glmnet

A second use case replaces the one-dimensional chain by a two-dimensional spatial graph. We generated an 8×8 grid of groups with 4-neighbor adjacency, $p = 16$ predictors, and two contiguous hotspot regions whose coefficient vectors differ from the background. Each group has 25 training observations for fitting, 10 validation observations for tuning, and 70 test observations. This setting mimics county-, sensor-, image-, or station-level regression problems where local regions share regression structure and boundaries are sparse.

The pooled `glmnet` baseline again cannot represent local heterogeneity. The group-specific `glmnet` baseline can represent heterogeneity, but without a spatial fusion penalty it produces fragmented coefficient maps and many spurious graph boundaries. The fused L1 model uses the grid graph to borrow strength across neighboring groups while preserving sharp region boundaries. In this controlled example it has the lowest test RMSE, the lowest coefficient error, and recovers the true hotspot boundaries without extra large graph-edge jumps.

6.3 L1 Accuracy and Efficiency

Figure 5 compares L1 test RMSE against the legacy L1 baseline. The exact operator method matches the legacy prediction error to numerical precision across the completed benchmark summary: the maximum absolute percent difference in mean RMSE is below 10^{-5} percent. This is expected because the operator form is an exact representation of the same active-edge objective.

The chain-specialized method is faster but approximate. Its median RMSE increase relative to the legacy method is 2.02% in the completed L1 summary, with larger losses on some datasets. This behavior is consistent with the method's design: replacing a graph by a chain changes the fusion prior.

Table 4: Synthetic structured-heterogeneity comparisons against `glmnet` baselines. The relative coefficient error is Frobenius error divided by the true coefficient Frobenius norm. CP distance is mean distance from true changepoints to the nearest estimated large jump. Boundary F1 is computed on graph edges by thresholding adjacent coefficient-vector differences.

Example	Method	Test RMSE	Coef. rel. error	Structure metric
Changepoint	pooled `glmnet`	1.957	0.848	CP dist. 28; 0 jumps
Changepoint	group `glmnet`	0.915	0.279	CP dist. 0; 59 jumps
Changepoint	fused L1	0.708	0.079	CP dist. 0; 2 jumps
Spatial hotspot	pooled `glmnet`	1.417	0.845	boundary F1 0.000; 0 jumps
Spatial hotspot	group `glmnet`	0.986	0.443	boundary F1 0.325; 103 jumps
Spatial hotspot	fused L1	0.813	0.196	boundary F1 1.000; 20 jumps

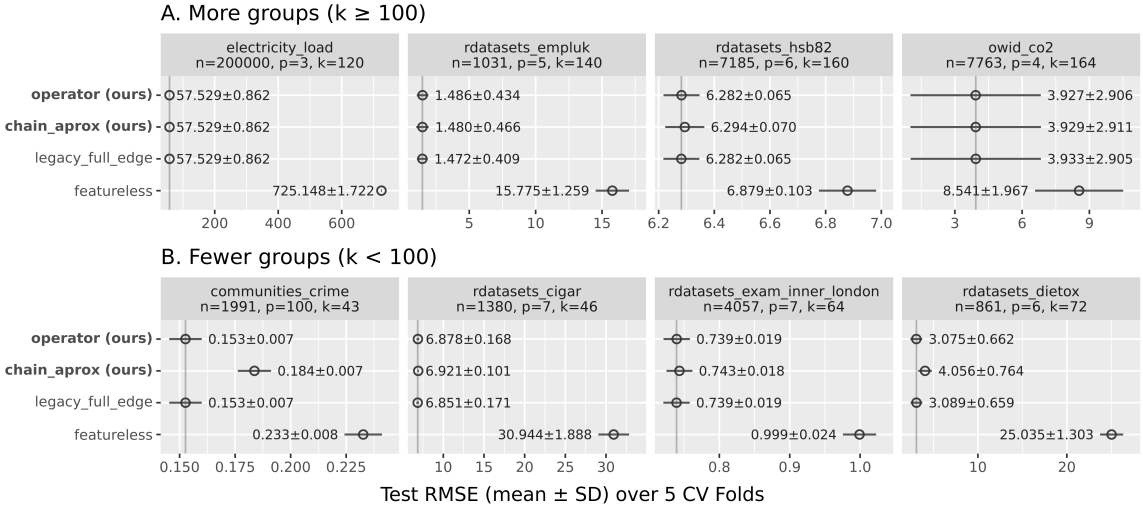

Figure 5: L1 prediction accuracy relative to the legacy L1 solver on the main benchmark panel. The exact operator solver preserves the legacy RMSE, while the chain-specialized approximation trades graph fidelity for speed.

Figure 6 summarizes L1 fit time across group-count regimes. In the completed L1 summary, the exact operator method has a median speedup of 2.26x over the legacy implementation, with speedups ranging from 0.68x to 7.65x. The one slowdown occurs on a small benchmark where overhead dominates the savings from active-edge representation. Median peak memory decreases by 10.7%.

The chain-specialized approximation has a median runtime speedup of 3.26x. Its runtime gains are larger because it always uses only $k - 1$ fusion edges. Its median peak-memory reduction is 4.6%, but memory gains vary by dataset because the implementation still carries fixed R and matrix overheads.

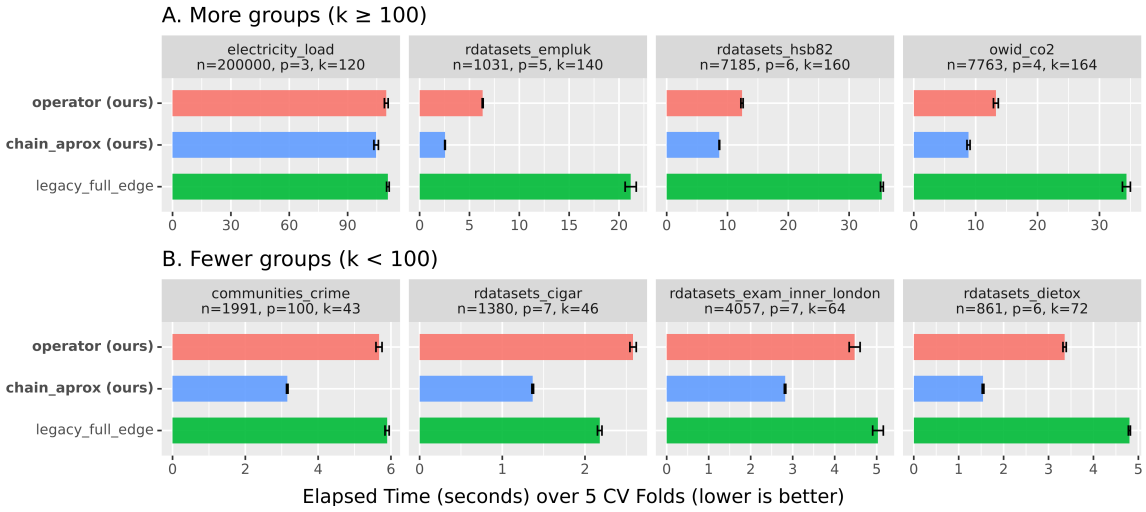

Figure 6: L1 fit time by solver and group-count regime. Active-edge methods reduce the cost of graph fusion when the graph is sparse.

Table 5: Selected L1 benchmark rows from the completed HPC summary. RMSE and fit time are means across five outer folds; memory is mean fit-worker peak RSS in MB.

Dataset	Method	RMSE	Time (s)	Memory (MB)
electricity_load	old_l1	57.5290	90.148	290.0
electricity_load	operator	57.5290	132.474	251.2
electricity_load	chain	57.5292	84.595	268.3
owid_co2	old_l1	3.9265	31.104	328.1
owid_co2	operator	3.9265	9.450	253.3
owid_co2	chain	3.9763	9.136	260.5
rdatasets_crime4	old_l1	0.0062	13.208	257.7
rdatasets_crime4	operator	0.0062	6.516	253.7
rdatasets_crime4	chain	0.0064	2.772	254.2
world_bank_wdi	old_l1	3478.8571	83.561	454.4
world_bank_wdi	operator	3478.8571	10.930	252.8
world_bank_wdi	chain	3478.8571	15.758	266.3

6.4 L2 Accuracy and Efficiency

The L2 active-edge builder is designed to preserve the legacy augmented objective while avoiding all-pair construction. Across the 19-dataset L2 summary, `new_l2` has median percent RMSE difference of -0.025% relative to `old_l2`. The largest observed RMSE increase is 0.134%, while several datasets show small decreases in RMSE from the active-edge construction and numerical path. The median fit-time speedup is 14.9x across all 19 datasets

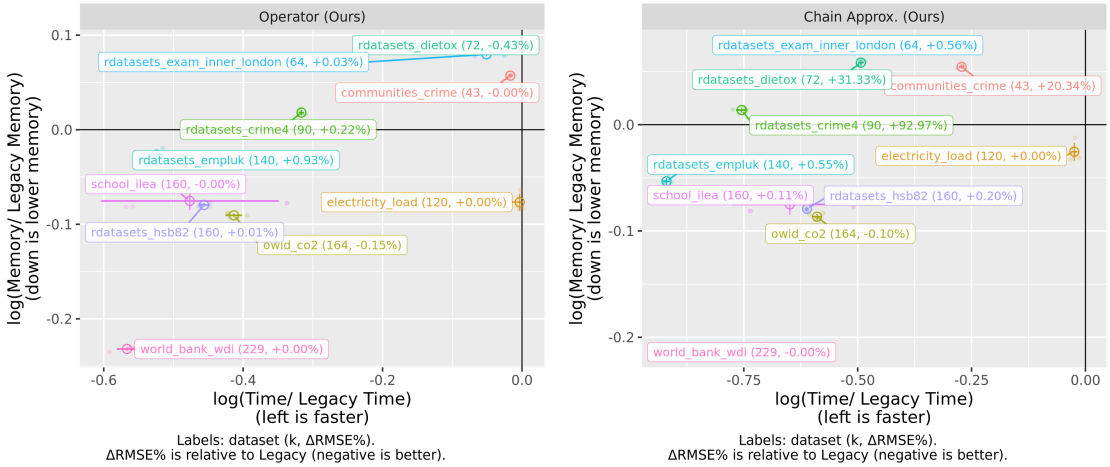

Figure 7: L1 efficiency relative to the legacy solver. Left: exact operator. Right: chain-specialized approximation.

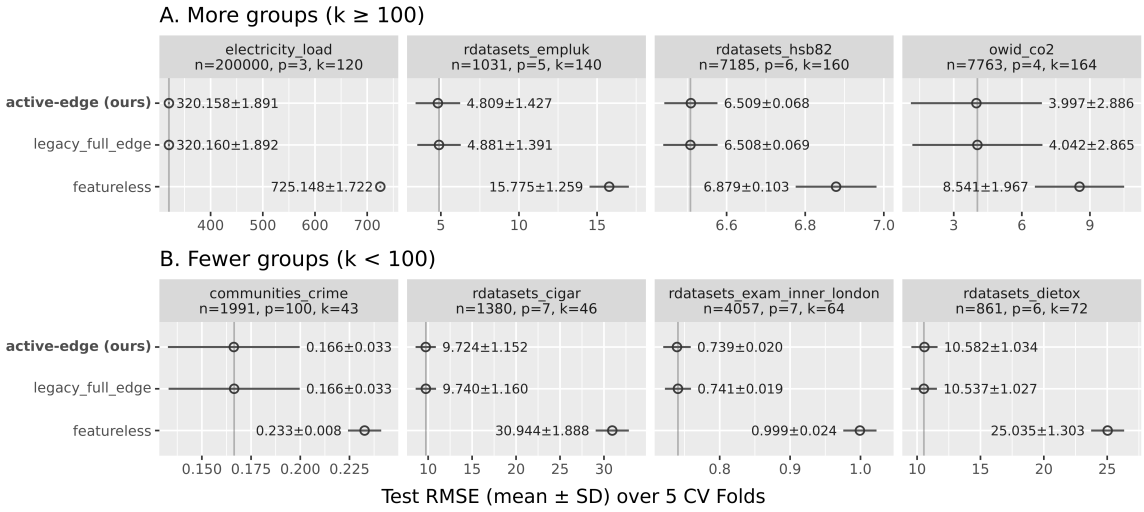

Figure 8: L2 prediction accuracy relative to the legacy L2 augmented-design solver across the benchmark panel.

and 105.3x on the 11 datasets with at least 50 groups. Median peak fit-worker memory decreases by 19.9% across the full L2 panel.

6.5 Synthetic Scaling Experiments

The repository also includes synthetic scaling experiments generated by the `atime` benchmark scripts. These experiments isolate solver scaling under controlled changes to the number of groups and graph structure. They support the same qualitative conclusion as

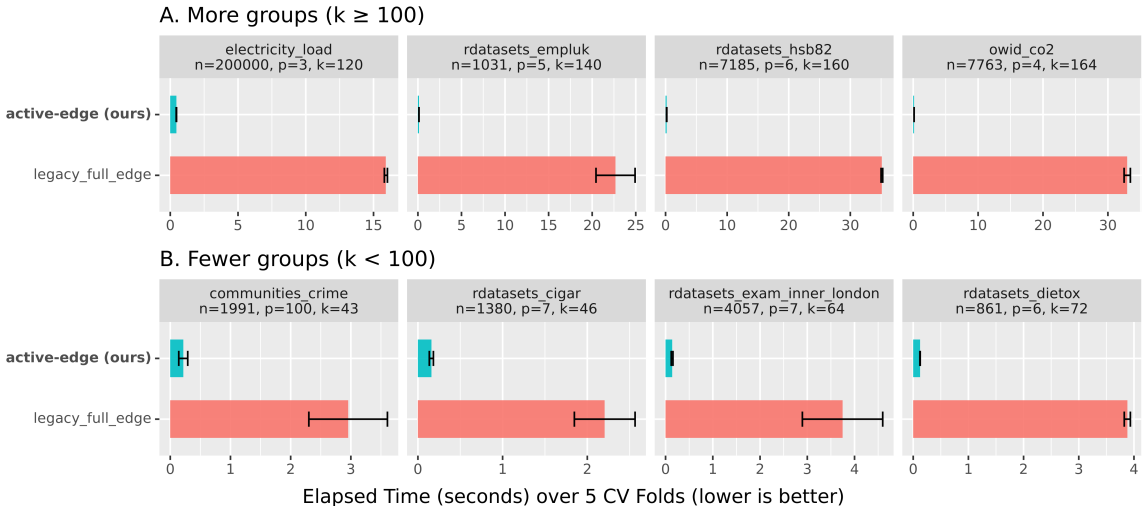

Figure 9: L2 fit time by solver and group-count regime. The active-edge builder reduces augmented-design construction when the graph is sparse.

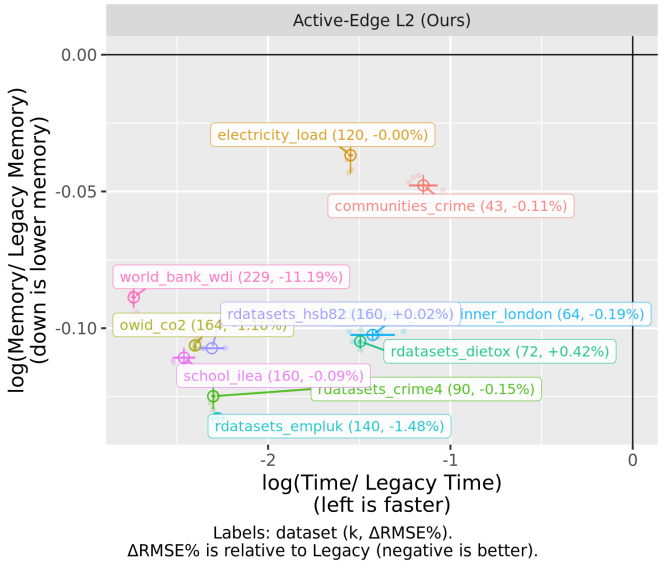

Figure 10: L2 efficiency of active-edge construction relative to the legacy L2 construction.

the real-data benchmarks: avoiding all-pair graph construction matters most when k grows and the graph remains sparse.

7 Discussion

The main lesson is representational: graph-aware statistical models should carry graph sparsity into the optimizer. When the graph has m edges, an implementation that constructs

$O(k^2)$ pairwise constraints can lose the computational advantage of modeling with a sparse graph. This is a general issue for edge-penalized regression, not only for `fuserplus`. Any method whose penalty decomposes over graph edges can pay an unnecessary quadratic cost if the graph is represented by a complete set of candidate pairs. The L1 operator and L2 active-edge builders recover the intended $O(pm)$ fusion representation while preserving the original objective in the exact cases.

The exact L1 operator is the safest default among the proposed L1 variants. It preserves the same fitted objective as the legacy active-edge formulation and gives substantial median runtime and memory improvements on the completed benchmark summary. The chain-specialized solver is more aggressive. It is best viewed as an approximation for exploratory or large-scale settings where a small loss in predictive accuracy is acceptable.

The scope of the contribution is computational rather than statistical, but it is not merely a local code optimization. The objectives come from the fused-lasso and graph-guided regression literature; the contribution is a solver representation that aligns the computational graph with the modeling graph. Edge-explicit L1 fusion updates, blocked edge processing, practical working-set expansion, DFS/MST chain approximation, and active-edge L2 matrix construction are instances of that representation. Propositions 1 and 2 identify the exact cases where the new representations preserve the legacy objective. The chain-specialized method is separated from those exact methods because replacing a graph by a chain changes the fusion geometry and should be interpreted as an approximation.

The empirical evidence is strongest for the exact L1 operator and active-edge L2 construction. The working-set and chain variants are included because they are useful solver modes in the package, but they involve additional choices about screening, edge expansion, or graph approximation. For that reason, the recommended default for sparse L1 fusion is the exact active-edge operator, with chain-specialized fitting reserved for exploratory or memory-constrained settings.

There are three important limitations. First, the paper analyzes representation equivalence and graph-construction complexity, but it does not introduce new statistical guarantees for fused regression estimators. The statistical behavior is inherited from the underlying fused-lasso and graph-guided regression objectives. Second, the largest gains occur when the user-supplied or data-derived graph is sparse. If the modeling graph is close to complete, the active-edge representation has little edge-count advantage. Third, the software is an R implementation, so small problems can be dominated by fixed interpreter, allocation, and `glmnet` overheads. These limits explain the occasional small-benchmark slowdown and motivate the recommendation to use the exact operator as the default sparse-graph path rather than as a universal speed guarantee.

8 Reproducibility

The implementation is in the `fuserplus`R package, version 0.1.0 (`DESCRIPTION`), evaluated from repository commit `99c69ae32a93e018852c6facc2d3f1e27a1068c5`. The benchmark environment used R version 4.5.2 (2025-10-31). The package contains R documentation under `man`, a vignette under `vignettes`, and `testthat` tests for the legacy L1/L2 solvers, Rcpp consistency, eigenvalue helpers, and the new L2 builder. The exported namespace includes the legacy user-facing solvers and helpers, while the benchmarked active-edge rou-

tines are implemented as package functions and invoked by the benchmark wrappers. The solver routines discussed in this paper are:

- `fusedLassoProximal`: legacy L1 fusion.
- `fusedLassoProximalNewOperator`: active-edge L1 fusion.
- `fusedLassoProximalChainSpecialized`: chain L1 approximation.
- `fusedL2DescentGLMNet`: legacy L2 fusion.
- `fusedL2DescentGLMNetNew`: active-edge L2 fusion.

Benchmark scripts are stored under `scripts/benchmark_*.R` and `scripts/hpc`. Processed dataset metadata and licensing notes are stored under `data/processed`. The HPC scripts include a manifest builder, a single-row runner, an aggregator, memory-tier splitting, and a SLURM array template using one CPU per task with conservative 24-hour and 32GB defaults. The exact CSV files used for the results in this paper are:

- `build/hpc/final_test_summary.csv` for the 12-dataset L1 panel.
- `build/hpc/l2_knn/final_test_summary.csv` for the 19-dataset L2 panel.
- `build/hpc/l2_knn/final_test_summary_k_ge_50.csv` for the large-group L2 subset.
- `build/changepoint/fused_vs_glmnet_summary.csv` for the synthetic changepoint comparison.
- `build/spatial_hotspot/fused_vs_glmnet_summary.csv` for the synthetic spatial-hotspot comparison.

The manifest-driven benchmark pipeline can be rerun with commands of the following form:

```
Rscript scripts/hpc/build_test_manifest.R \
  --objective l1 \
  --methods old_l1,operator,chain_specialized \
  --out_csv build/hpc/test_manifest.csv \
  --results_dir build/hpc/test_jobs

Rscript scripts/hpc/run_test_job.R \
  --manifest_csv build/hpc/test_manifest.csv \
  --row_id 1

Rscript scripts/hpc/aggregate_test_results.R \
  --manifest_csv build/hpc/test_manifest.csv \
  --out_all_csv build/hpc/test_results_all.csv \
  --out_summary_csv build/hpc/final_test_summary.csv
```

The L2 panel uses the same pipeline with `--objective l2`, `--methods old_l2,new_l2`, and output paths under `build/hpc/l2_knn`. The synthetic changepoint comparison is generated by `scripts/benchmark_changepoint_fused_vs_glmnet.R`, and the spatial-hotspot comparison is generated by `scripts/benchmark_spatial_hotspot_fused_vs_glmnet.R`. The synthetic scaling experiments are generated by the `benchmark_atime_*` scripts under `scripts`. The paper source can be rebuilt from `publication/paper1/fuserplus_jmlr.tex` with the JMLR style file in the same directory.

9 Conclusion

`fuserplus`demonstrates that sparse-graph construction can materially improve grouped fused regression solvers without changing the statistical model. The exact L1 operator and active-edge L2 augmented-design builder preserve the legacy objectives while reducing avoidable all-pairs graph costs. The chain-specialized L1 approximation provides an additional high-efficiency mode when the user is willing to trade graph fidelity for speed. These changes make fused regression more practical for grouped datasets with many subgroups and sparse relationships.

Acknowledgments and Disclosure of Funding

Daniel Agyapong is affiliated with Northern Arizona University. The author thanks the maintainers of the public datasets and R packages used in the benchmark suite.

References

Vincent Arel-Bundock. Rdatasets: A collection of datasets originally distributed in R packages, 2026. URL `https://vincentarelbundock.github.io/Rdatasets/`.

Manuel Arellano and Stephen Bond. Some tests of specification for panel data: Monte carlo evidence and an application to employment equations. *The Review of Economic Studies*, 58(2):277–297, 1991. doi: 10.2307/2297968.

Taylor B. Arnold and Ryan J. Tibshirani. *`genlasso`: Path Algorithm for Generalized Lasso Problems*, 2025. URL `https://cran.r-project.org/package=genlasso`. R package reference manual.

Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/2200000016.

Song Chen. Beijing multi-site air-quality data. UCI Machine Learning Repository, 2017. DOI: 10.24432/C5RK5G.

Xi Chen, Qihang Lin, Seyoung Kim, Jaime G. Carbonell, and Eric P. Xing. Graph-structured multi-task regression and an efficient optimization method for general fused

lasso. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1–10, 2010.

Xi Chen, Qihang Lin, Seyoung Kim, Jaime G. Carbonell, and Eric P. Xing. Smoothing proximal gradient method for general structured sparse regression. *The Annals of Applied Statistics*, 6(2):719–752, 2012a.

Xiaohui Chen. *MMT: A Matlab Library for Multi-task Learning*, 2012. URL `https://people.ece.ubc.ca/xiaohuic/code/multi-task_lasso/mmt_user_manual.pdf`. User manual.

Xiaohui Chen, Xinghua Shi, Xing Xu, Zhiyong Wang, Ryan Mills, Charles Lee, and Jinbo Xu. A two-graph guided multi-task lasso approach for eQTL mapping. In *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics*, volume 22 of *Proceedings of Machine Learning Research*, pages 208–217, 2012b.

Christopher Cornwell and William N. Trumbull. Estimating the economic model of crime with panel data. *The Review of Economics and Statistics*, 76(2):360–366, 1994.

Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016. URL `https://www.cvxpy.org/`.

Frank Dondelinger and Sach Mukherjee. *`fuser`: Fused Lasso for High-Dimensional Regression over Groups*, 2017. URL `https://cran.r-project.org/package=fuser`. R package and vignette.

Frank Dondelinger, Sach Mukherjee, and The Alzheimer's Disease Neuroimaging Initiative. The joint lasso: High-dimensional regression for group structured data. *Biostatistics*, 21 (2):219–235, 2020. doi: 10.1093/biostatistics/kxy035.

Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010.

Suman Ghosh and Nilanjan Chatterjee. *`extlasso`: Extended Lasso Penalized Regression*, 2026. URL `https://cran.r-project.org/package=extlasso`. R package reference manual.

Jelle J. Goeman. *`penalized`: L1 and L2 Penalized Estimation in GLMs and in the Cox Model*, 2026. URL `https://cran.r-project.org/package=penalized`. R package reference manual.

Harvey Goldstein, Jon Rasbash, Min Yang, Geoffrey Woodhouse, Huiqi Pan, Desmond Nuttall, and Sally Thomas. A multilevel analysis of school examination results. *Oxford Review of Education*, 19(4):425–433, 1993. doi: 10.1080/0305498930190401.

David Hallac, Jure Leskovec, and Stephen Boyd. Network lasso: Clustering and optimization in large graphs. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 387–396, 2015.

Holger Hoefling. A path algorithm for the fused lasso signal approximator. *Journal of Computational and Graphical Statistics*, 19(4):984–1006, 2010.

Holger Hoefling. *flsa: Path Algorithm for the General Fused Lasso Signal Approximator*, 2024. URL `https://cran.r-project.org/package=flsa`. R package reference manual.

Joseph D. Janizek and Su-In Lee. `yaglm`: A python package for fitting and tuning generalized linear models that supports structured, adaptive and non-convex penalties, 2021. URL `https://github.com/yaglm/yaglm`.

JuliaStats. *Lasso.jl: Lasso, Fused Lasso, and Trend Filtering Documentation*, 2026. URL `https://juliastats.org/Lasso.jl/stable/smoothing/`.

Charlotte Lauridsen, Søren Højsgaard, and Martin Tang Sørensen. Influence of dietary rapeseed oil, vitamin E, and copper on the performance and the antioxidative and oxidative status of pigs. *Journal of Animal Science*, 77(4):906–916, 1999. doi: 10.2527/1999.774906x.

Yurii Nesterov. Smooth minimization of non-smooth functions. *Mathematical Programming*, 103(1):127–152, 2005.

New York City Taxi and Limousine Commission. 2022 yellow taxi trip data. NYC Open Data, 2022. URL `https://data.cityofnewyork.us/Transportation/2022-Yellow-Taxi-Trip-Data/qp3b-zxtp`.

Nooshin Omranian, Jeanne M. O. Eloundou-Mbebi, Bernd Mueller-Roeber, and Zoran Nikoloski. Gene regulatory network inference using fused LASSO on multiple data sets. *Scientific Reports*, 6:20533, 2016. doi: 10.1038/srep20533.

OpenML. Sarcos: Openml dataset 43873, 2026. URL `https://www.openml.org/d/43873`.

Oscar Hernan Madrid Padilla, James Sharpnack, James G. Scott, and Ryan J. Tibshirani. Adaptive nonparametric regression with the K-nearest neighbour fused lasso. *Biometrika*, 107(2):293–310, 2020.

Ashley Petersen, Daniela Witten, and Noah Simon. Fused lasso additive model. *Journal of Computational and Graphical Statistics*, 25(4):1005–1025, 2016.

José C. Pinheiro and Douglas M. Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, New York, 2000. doi: 10.1007/b98882.

PyMC Development Team. Pymc examples, 2026. URL `https://github.com/pymc-devs/pymc-examples`.

Stephen W. Raudenbush and Anthony S. Bryk. *Hierarchical Linear Models: Applications and Data Analysis Methods*. Sage Publications, Thousand Oaks, CA, 2 edition, 2002.

Michael Redmond. Communities and crime. UCI Machine Learning Repository, 2002. DOI: 10.24432/C53W3X.

Stephen Reid and Robert Tibshirani. *smoothedLasso: A Framework to Smooth L1 Penalized Regression Operators Using Nesterov Smoothing*, 2021. URL `https://cran.r-project.org/package=smoothedLasso`. R package reference manual.

Hannah Ritchie, Pablo Rosado, and Max Roser. CO2 and greenhouse gas emissions. Our World in Data, 2023.

Wesley Tansey, Yu-Xiang Wang, David M. Blei, and Raul Rabadan. The DFS fused lasso: Linear-time denoising over general graphs. *Journal of Machine Learning Research*, 19 (176):1–36, 2018.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.

Robert Tibshirani, Michael Saunders, Saharon Rosset, Ji Zhu, and Keith Knight. Sparsity and smoothness via the fused lasso. *Journal of the Royal Statistical Society: Series B*, 67 (1):91–108, 2005.

Ryan J. Tibshirani and Jonathan Taylor. The solution path of the generalized lasso. *The Annals of Statistics*, 39(3):1335–1371, 2011.

Artur Trindade. Electricityloaddiagrams20112014. UCI Machine Learning Repository, 2015. DOI: 10.24432/C58C86.

Yu-Xiang Wang, James Sharpnack, Alexander J. Smola, and Ryan J. Tibshirani. Trend filtering on graphs. *Journal of Machine Learning Research*, 17(105):1–41, 2016.

World Bank. World development indicators. World Bank DataBank, 2026.

Gui-Bo Ye and Xiaohui Xie. Split bregman method for large scale fused lasso. *Computational Statistics & Data Analysis*, 55(4):1552–1569, 2011. doi: 10.1016/j.csda.2010.10.021.

Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B*, 68(1):49–67, 2006.

Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2):301–320, 2005. doi: 10.1111/j.1467-9868.2005.00503.x.